Linguaggi di programmazione

Francesco Lanzarini

1 Capitolo 1 - Macchine astratte, interpreti, compilatori

1.1 Che cosa è una macchina astratta? E in cosa si differenzia da una macchina fisica?

Una macchina astratta e' un insieme di strutture dati e algoritmi in grado di memorizzare e esegiure programmi, la parte esseziale di essa e' l'interprete. Una MA si differenza da una macchina fisica perche' e' capace, tramite alcune fasi, di eseguire programmi scritti in linguagggi diversi; infatti una macchina fisica e' in grado di eseguire programmi scritti nel suo linguaggio.

1.2 Che cos'è un interprete? In cosa consiste il ciclo fetch-decode-execute?

L'interprete e' il componente essenziale di una macchina astratta ed ha il compito di interpretare le istruzioni, esso e' costituito da: operazioni per elaborare istruzioni primitive, operazioni e strutture dati per controllare la sequenza di esecuzione, trasferimento dati e gestione memoria. Il ciclo fde consiste nel prelevare la giusta istruzione da eseguire (fetch), capire quale operzione e' necessario eseguire (decode) e eseguirla salvando eventualmente il risultato in una certa zona di memoria (execute).

1.3 Cos'è il linguaggio macchina?

Il linguaggio macchina e' il linguaggio che la macchina e' in grado di comprendere ed eseguire.

1.4 Possono esistere macchine diverse con lo stesso linguaggio macchina?

Si, perche' la stessa istruzione elementare puo' essere implementata in modo differente in due macchine diverse.

1.5 In quali modi è possibile implementare una macchina astratta? Elencare vantaggi e svantaggi delle varie tecniche.

Sono tre i principali modi per realizzare una macchina astratta:

1. Hardware: e' teoricamente sempre possibile ed ha la velocita' di esecuzione massima ma non e' per nulla flessbile per cui e' usata solo per macchine di basso livello o dedicate
2. Microprogrammi in ROM: rende microprogrammabile la macchina ospite, ha una maggiore falessibilita' ma minore velocita'.
3. Software di iterpretazione o simulazione: ovvero strutture dati e algoritmi della macchina astratta scritti in lunguaggio della macchina ospite, puo' essere implementata in qualsiasi macchina ospite, massima flessibilita' ma minima velocita'.

Nella pratica in realta' si usa una combinazione di queste tecniche.

1.6 Che cos'è un compilatore?

Il compilatore e' un programma che traduce un programma da una lingua L_1 ad una lingua L_0 preservandone la semantica.

1.7 Relativamente alla tecnica d'implementazione software, descrivere la tecnica d'implementazione interpretativa pura e quella compilativa pura.

Quando si esegue un'implementazione interpretativa pura si realizza un interprete scritto in L_0 (linguaggio macchina) per un linguaggio generico L, le singole istruzioni sono interpretate e poi eseguite. Mentre in un'implementazione compilativa pura i programmi scritti in L sono tradotti in programmi equivalenti in L_0 e poi eseguiti sulla macchina fisica

1.8 Quando un interprete si può dire corretto? Quando un compilatore si può dire corretto?

Un interprete si puo' dire corretto quando realizza una funzione parziale: $I_L^{L_0} : (P_r^L \times D)$ tale che $I_L^{L_0}(P_r^L, Input) = P^L(Input)$. Un compilatore si puo' dire corretto quando realizza una funzione totale: $C_{L,L_0} : Prog^L \to Prog_0^L$ tale che $C_{L,L_0}(P_r^L) = P_r c^{L_0}$

1.9 Confrontare l'implementazione di una macchina astratta su una macchina ospite per mezzo di un interprete o di un compilatore.

In un'implementazione interpretativa pura si ha una scarsa efficienza della macchina astratta perche' bisogna sommare ai tempi di esecuzione i tempi di decodifica del codice, inoltre nel caso una stessa istruzione dovesse ripetersi l'interprete deve decodificarla piu' volte; essa e' pero' un implementazione piu' flessibile, piu' facile da realizzare e non genera altro codice da memorizzare. Mentre un'implementazione comilativa pura puo' vantare un'ottima velocita' visto che tutto il codice e' gia' tradotto, essa e' pero' piu' difficile da realizzare inoltre e' poco flesssibile (bisognerebbe riscrivere il compilatore), astrae di piu' dal programma sorgente e occupa piu' memoria.

1.10 Come vengono implementate nella realtà le macchine astratte? Che cos'è la macchina intermedia?

Nella realta' e un misto delle due interpretazioni: di solito si compila il programma in una lingua intermedia L_i per una la macchina intermedia, suiccessivamente un interprete $I_{L_i}^{L_0}$ interpreta le istruzioni per l'esecuzione. Generalmente $I_{L_i}^{L_0}$ = interprete della macchina ospite + opportuni meccanismi chiamati supporto a run tim (SRT).

1.11 Quando si dice che una implementazione è di tipo interpretativo e quando di tipo compilativo? Fare esempi di linguaggi la cui implementazione è di un tipo o dell'altro.

Solitamente si dice che un implementazione e' interpretativa quando la maggior parte delle istruzioni vengono interpretate e poi eseguite, tipo in python e Java; mentre un'implementazione e' di tipo compilativo quando la maggior parte delle istruzioni sono prima tradotte e poi eseguite, tipo in C e C++.

1.12 Cos'è la Just-in-Time compilation?

La JIT e' una combinazione di AOT (ahead of time) compilation e interpretazione: si genera prima un codice intermedio (byte code in java) che viene poi eseguito passando per un compilatore on-demand man mano che esegue il programma tradotto in linguaggio macchina.

1.13 L'interprete e il compilatore si possono sempre realizzare?

Non sempre si possono realizzare, infatti il linguaggio di destinazione deve essere espressivo almeno quanto quello di partenza perchè se no alcune funzioni non sarebbero traducibili.

1.14 Che cosa è l'implementazione via kernel?

L'implementazione via kernel è un'implementazione che dato il linguaggio L prevede di individuare un sotto-insieme H, si scrive un compilatore in H che traduca da L a L_0 o un interprete in H per L e successivamente un compilatore/interprete per H. Questo fa si che si debbano scrivere due programmi ma essendo più vicini sia un lavoro più semplice e garantendo maggiore portabilità.

1.15 Cosa si intende per bootstrapping?

Per bootstrapping si intende una tecnica che consiste nell'utilizzare compilatori o interpreti per diverse lingue per tradurre un programma in un'altra lingua. Per esempio si consideri un compilatore che traduce da A a B e un altro da B a C, se volessimo tradurre un programma scritto in A nella lingua C possiamo usare i due compilatori al posto di crearne uno nuovo.

2 Capitolo 2 - Descrivere un linguaggio di programmazione

2.1 Quali sono i livelli di descrizione di un linguaggio?

I livelli di descrizione di un linguaggio sono quattro:

1. Sintassi: regole per cui una frase e' corretta.
2. Semantica: regole per cui una frase ha senso.
3. Pragmatica: il modo in cui sono usate le frasi corrette e sensate.
4. Implementazione: solo per i linguaggi eseguibili, come eseguire una frase rispettandone la semantica.

2.2 Sintassi: qual è l'aspetto lessicale e quale quello grammaticale di un linguaggio?

L'aspetto lessicale di un lunguaggio e' quello che specifica le parole che si possono usare, mentre quello grammaticale specifica come frasi corrette si possono ottenere a partire dal lessico.

2.3 Cos'è un alfabeto? Cos'è una parola o stringa? Cos'è A*? Tale insieme è enumerabile?

Un alfabeto e' un insieme tipicamente finito di simboli o caratteri, una parola o stringa e' una sequenza dinita di caratteri dell'alfabeto. Un linguaggio formale L su un alfabeto A e' un sottoinsieme di $A*$, $A*$ e' l'insieme di tutte le possibili parole sull'alfabeto A. $A*$ e' un infinito contabile, quindi e' contabile.

2.4 Definizione di potenza di una stringa. Definizione di potenza di un linguaggio. Definizione di chiusura/iterazione (o stella di Kleene) di un linguaggio.

- Potenza di una stringa: Data w una stringa, la sua potenza e' definita come: $w^0 = \epsilon$, $w^{n+1} = ww^n$
- Potenza di un linguaggio: Dato L un linguaggio, la sua potenza e' definita come: $L^0 = \{\epsilon\}$, $L^{n+1} = L \cdot L^n$
- Chiusura/iterazione (o stella di Kleene) di un linguaggio: $L* = Un \geq 0 L^n$, $L^+ = Un \geq 1 L^n$ (chiusura positiva)

2.5 Definizione di grammatica libera da contesto. Come si deriva una stringa? Qual è il linguaggio generato da una grammatica libera?

Una grammatica libera da contesto e' una quadrupla (NT,T,R,S) tale che:

- NT: e' l'insieme finito di non terminali.
- T: e' l'insieme finito di terminali.
- A $\in$ NT: e' il simbolo iniziale.
- R: e' l'insieme finito di produzioni della forma $V \to \alpha$

Per derivare una stringa si parte da P (non terminale, per esempio $P \to \epsilon|a|b|aPa|bPb$) e si applicano le produzioni. Il linguaggio generato da una grammatica libera e' l'insieme: $L(G) = \{w \in T | S \Rightarrow^* w\}$

2.6 Cos'è un albero di derivazione? Cos'è una derivazione canonica sinistra/destra (leftmost/rightmost)? Esiste una corrispondenza biunivoca tra alberi di derivazioni e derivazioni canoniche?

L'albero di derivazione e' un albero binario associato ad una particolare derivazione: esso e' ordinato e ogni nodo e' etichettato con un simbolo. Per derivazione canonica destra/sinistra si intende una derivazione che ad ogni passo va a riscrivere il non-terminale piu' a destra/sinistra. Esiste inoltre una corrispondenza biunivoca tra derivazioni leftmost/rightmost, piu' precisamente un albero di derivazione esiste una sola derivazione che lo genera.

2.7 Quando una grammatica è ambigua? (fare un esempio) Quando un linguaggio è ambiguo? (fare un esempio)

Una grammatica e' ambigua se esiste una stringa che ammette piu' alberi di derivazione, mentre un linguaggio ambiguo se tutte le grammatiche G tali che $L(G) = L$ sono ambigue.

2.8 È possibile rimuovere l'ambiguità dalla grammatica delle espressioni aritmetiche? Come?

Si, e' possibile introducendo una precedenza tra segni e l'associativita' degli stessi.

2.9 Cos'è l'albero di sintassi astratta? Che differenza c'è tra sintassi concreta e sintassi astratta? Cos'è lo zucchero sintattico?

L'albero di sintassi astratta e' un albero di derivazione da sintassi concreta senza zucchero sintattico. La sintassi concreta e' una sintassi piu' semplice ma ambigua, mentre la sintassi astratta e' piu' complessa ma non ambigua e con uso di zuccero sintattico, ovvero termini ausialiari usati per disanbiguare ma sintatticamente irrilevanti.

2.10 Fare esempi di vincoli sintattici contestuali. Possono essere catturati attraverso grammatiche libere?

Degli esempi di vincoli sintattici contestuali sono le dichiarazioi di variabili, type checking, corrispondenza tra dichiarazione e chiamata di funzione, visibilita' e scope, unicita' dei nomi e correttezza del flusso. Non possono essere catturati attraverso grammatiche libere.

2.11 Cosa s'intende per semantica statica? E per semantica dinamica?

Per semantica statica si intendono tutti i vincoli che possono essere verificati ispezionando il programma prima di mandarlo in esecuzione mentre quella dinamica considera il significato delle istruzioni eseguite.

2.12 Elencare le varie fasi in cui si articola un compilatore. Descrivere in dettaglio ogni singola fase.

Le fasi in cui si articola un compilatore sono 6:

1. **Analisi Lessicale**: spezza il programma sorgente in token, controlla se il lessico sia ammissibile (se le parole sono scritte bene ed esistono) e riempie parzialmente la tabella dei simboli.
2. **Analisi Sintattica**: verifica che le frasi siano corrette a partire dalla lista di token e da in output l'albero di derivazione del programma.
3. **Analizzatore semantico**: esegue controlli di semantica statica e arricchisce l'albero di derivazione.
4. **Generazione della forma intermedia**: genera del codice scritto in un linguaggio intermedio che e' facilmente traducibile in linguaggio macchina, per generarlo segue la struttura dell'albero di derivazione.
5. **Ottimizzatore**: Ottimizza il programma scritto in pseudo codice eliminando operazioni inutili, pezzi di codice inutili ecc.
6. **Generazione del codice**: a questo punto il codice ottimizzato nel linguaggio intermedio viene effettivamente tradotto in linguaggio macchina.

2.13 A chi serve definire la semantica dinamica di un linguaggio e perché?

La semantica dinamica di un linguaggio serve al programmatore per sapere cosas fa il suo programma e per dimostrarlo, al progettista del linguaggio come specifica e per dimostrarne le proprieta' e all'implementatore del linguaggio come modo per dimostrare la corrrettezza dell'implementazione.

2.14 Quali tecniche si usano per dare semantica dinamica ad un linguaggio di programmazione?

Sono due le principali tecniche usate:

- Operazionale: attraverso una macchina astratta, quindi una specie di automa, che mostra come vengono eseguite le istruzioni.
- Denotazionale: ad ogni programma sequenziale si associa una funzione di input ad output, pone enfasi sul cosa si calcola.

2.15 Imparare le regole di semantica operazionale SOS per il semplice linguaggio presentato a lezione. Regole di valutazione: interna vs esterna, sinistra vs destra vs parallela.

2.16 Cosa s'intende per pragmatica? Cosa si intende per implementazione di un linguaggio?

Per pragmatica si intende un insieme di regole sul modo migliore per utilizzare le istruzioni del linguaggio di programmazione, mentre per implementazione si intende la vera e propria scrittura del compilatore per una macchina ospite, realizzando cosi' la macchina astratta.

3 Capitolo 3 - Analisi lessicale-Linguaggi regolari

3.1 Cosa fa l'analizzatore lessicale?

L'analizzatore lessicale riconosce nella stringa in ingresso gruppi/sequenze di simboli che corrispondono a specifiche categorie sintattiche, in output la stringa viene trasformata in una serie di token.

3.2 Cos'è un token?

Il token e' una coppia $< nome, valore >$, il nome e' un simbolo astratto che rappresenta una categoria sintattica mentre il valore e' al sequenza di simboli in ingresso.

3.3 Cos'è un pattern? Come lo si rappresenta? Cos'è un lessema?

Il pattern e' la descrizione generale della forma che possono avere i valori di una classe di token e lo si rappresenta tramite un'espressione regolare, mentr eil lessema e' una stringa istanza di un pattern.

3.4 Definizione di espressioni regolari (sintassi) e di linguaggio associato (semantica).

Fissato un alfabeto A, si definiscono le espressioni regolari su A le seguenti BNF:

$$r ::= \emptyset|\epsilon|a|r \cdot r|(r|r)|r^*$$

con a e' un simbolo di A, *cdot* e' l'operazione di concatenazione, | disgiunzione e $*$ la ripetizione o stella di Kleene. Il linguaggio associato e' la funzione $L : exp - reg \rightarrow P(A*)$ che prende un'espressione regolare e ritorna tutti i possibili linguaggi formali che ammettono quell'espressione.

3.5 Quali sono i linguaggi regolari? I linguaggi finiti sono tutti regolari? Esistono linguaggi infiniti regolari?

Un linguaggio e' definito regolare sse esiste un'espressione regolare r tale che L= $L[r]$, inoltre e' sicuro che ogni linguaggio finito sia regolare ed esistono linguaggi infiniti che sono regolari.

3.6 Definizione di equivalenza tra espressioni regolari. Elencare alcune leggi di equivalenza.

Due espressioni regolari r e s sono equivalenti sse $L[r] = L[s]$, le leggi si trovano a pagina 285 degli appunti.

3.7 Cos'è una definizione regolare e a cosa serve?

Se intende espressione regolare la definizione e' data prima e serve a specificare il pattern di una categoria sintattica.

3.8 Definizione di NFA (automa finito nondeterministico). Discutere cosa si intende per nondeterminismo. Mettere in relazione la definizione formale con la rappresentazione grafica come diagramma di transizioni.

Un NFA e' una quintupla $(\Sigma, Q, \delta, q_0, F)$ dove:

- Σ e' un alfabeto di simboli in input
- Q e' un insieme finito di stati
- $q_0 \in Q$ e' lo stato iniziale
- $F \subseteq Q$ e' l'insieme di stati finali
- δ e' la funzione di trasizione che si aspetta in input uno stato e un simbolo e in output restituisce i possibili stati raggiungibili.

Per non determinismo si intende la proprieta' di un automa di offrire due mosse alternative per lo stesso carattere a partire dallo stesso stato, L'ultima domanda si tratta di fare un disegnino.

3.9 Come si definisce il linguaggio accettato da un NFA? Definizione di equivalenza tra NFA.

Informalmente un NFA accetta w sse nel diagramma di transizione esiste un cammino da q_0 a F nel quale la stringa che si ottiene concatenando le etichette degli archi e' w, due NFA sono equivalenti sse riconoscono lo stesso linguaggio.

3.10 Definizione di DFA (automa finito deterministico). Discutere cosa si intende per determinismo. Mostrare che un DFA è un caso speciale di NFA, ovvero la classe dei DFA è un sottoinsieme della classe degli NFA.

Un DFA e' un quintupla definita esattamente come un NFA con la differenza nella funzione di transizione perche' l'insieme delle mosse possibili e' sempre un singoletto. Meno formalmente un DFA a differenza di un NFA ha una e una sola transizione possibili per ogni carattere a partire da ogni stato e non ha transizioni ϵ. Ne viene logicamente che $DFA \subset NFA$

3.11 Dato un NFA, come si ricava un equivalente DFA? Ovvero descrivere come è definita la costruzione per sottoinsiemi. Definizione di epsilon-closure e algoritmo associato. Qual è la complessità della costruzione per sottoinsiemi nel caso pessimo? Ovvero se NFA ha n stati, quanti stati può avere il DFA equivalente?

Per vedere come si fa guaradre da pagina 210 dei lucidi e appunti. Per $\epsilon - closure$ di q si intende l'insime di stati raggiungibili a partire da q con sole mosse ϵ. In altre parole per costruire lo stato di arrivo del DFA a partire dallo stato P e carattere b basta fare $\epsilon - closure(mossa(P, b))$ con mossa che ritorna lo stato in cui si arriva partendo da P e leggendo il carattere b. Algoritmo a pagina 314 di lucidi e appunti. Nel caso pesimo in cui stiamo costruendo il DFA dal NFA l'insieme degli stati del DFA sara' al massimo l'insieme delle parti dell'insieme degli stati dell'NFA; per cui detto $n = numero\ stati\ NFA$ il numero di stati del DFA sara' al massimo 2^n.

3.12 Dato i due punti precedenti, enunciare il teorema che dice che la classe dei linguaggi riconosciuto da NFA coincide con la classe dei linguaggi riconosciuti da DFA.

Il teorema a cui l'enunciato fa da corollario e':

Sia $N = (\Sigma, Q, \delta, q_0, F)$ un NFA e sia M_n l'automa ottenuto con la costruzione per sottoinsiemi. Allora M_N e' un DFA e si ha che

$$L[N] = L[M_N]$$

3.13 Come si costruisce un NFA a partire da una espressione regolare, in modo tale che il linguaggio riconosciuto dall'NFA sia lo stesso del linguaggio associato all'espressione regolare?

C'e' un teoema che ci assicura la possibilita' e slide di spiegazione da pagina 326 di lucidi e appunti.

3.14 Definizione di grammatica regolare.

Un grammatica libera(NT,T,R,S) e' definita regolare sse ogni produzione R r' della forma A→aB oppure A→a oppure A→ ϵ con A,BinNT e A∈T. In pratica in una forma sentenziale generata puo' occorrere al piu' un terminale che si trova nella posizione piu' a destra, quindi e' ammesso A→aA ma non A→aA.

3.15 Come si associa ad una grammatica regolare un equivalente NFA?

Graze al teorema di pagina 341 di lucidi e appunti.

3.16 Dato un DFA, come si costruisce una grammatica regolare equivalente?

Per associare al DFA la grammatica equivalente basta prendere ogni stato guardare i caratteri uscenti e metterci in coda lo stato a cui portano (esempio pagina 344)

3.17 Data una grammatica regolare, come si costruisce un'espressione regolare equivalente?

Data una grammatica regolare e' sufficiente trattarla come un'espressione per ricavare ogni volta il non terminale piu' in basso e maneggiare le espressioni con le equivalenze per ottenere un'espressione regolare che e' equivalente.

3.18 Descrivere, con un diagramma riassuntivo, tutte le relazioni fra i formalismi introdotti: NFA, DFA, grammatiche regolari, espressioni regolari. Questo diagramma dimostra che tutti questi formalismi sono equivalenti e descrivono la classe dei linguaggi regolari.

Pagina 351 lucidi e appunti.

3.19 Definizione di stati equivalenti in un DFA. Quando due stati di un DFA sono distinguibili?

Due stati sono equivalenti in un DFA se il linguaggio riconosciuto dall'NFA partendo dai due stati è il medesimo. Due stati invece non sono equivalenti se il linguaggi riconosciuti a partire dai due stati non sono differenti, ovvero se uno dei due arriva ad un non terminale e l'altro ad un terminale.

3.20 Come funziona l'algoritmo iterativo con tabella a scala per produrre le classi di equivalenza di stati di un DFA?

Pagina 368 lucidi e appunti.

3.21 Una volta determinate le classi di equivalenza degli stati di un DFA, come si costruisce l'automa minimo associato?

Si raggruppano in un unico stato gli stati equivalenti e si riscrive l'automa nella sua forma minima.

3.22 Cos'è Lex? Qual è il suo input e il suo output?

Lex e' il capostipite dei programmi generatori di scanner, in input prende un file .l che contiene una serie di pattern e le azioni ad essi associati e in output realizza un automa riconoscitore in C che associa ad ogni istanza del pattern l'azione relativa.

3.23 Qual è la struttura di un file .l? Come funziona l'analizzatore lessicale prodotto da Lex?

Il file .l e' strutturato in tre parti:

1. Dichiarazioni: cioe' le definizioni regolari
2. Regole: ovvero un dupla espressione regolare (pattern) azione (scritto in C)
3. Funzioni ausialiari: definizioni di possibili funzioni complesse che si usano nell'azione.

Il programma prodotto da Lex implementa il DFA che riconosce le espressioni regolari definite dalle regole per cui scandisce il testo sorgente e quando riconosce un lessema esegue l'azione ad esso associato e passa in output il risultato dell'azione; se invece l'input non corrisponde ad un poattern genera un segnale di errore.

3.24 Come si interfaccia Lex con Yacc?

A livello pratico lo scanner generato da Lex e' una sotto routine usata dal praser generato da YACC: quando il preser ha bisogno del token effettua una chiama allo scanner che a esecuzione terminata restituira il token. I due programmi per scambiarsi le informazioni hanno a disposizione variabili comuni.

3.25 Intestazione e dimostrazione del pumping lemma.

Sia L un linguaggio regolare, allora $\exists N > 0$ tale che $\forall z \in L$ con $|z| \geq N, \exists u, v, w$ tali che:

- $z = uvw$
- $|uv| \leq N$
- $|v| \geq 1$

- $\forall k \geq 0 : uv^k w \in L$

Dimostrazione: Sia $N = |Q_M|$ con M il DFA minimo che accetta L e $z = a_1 a_2 ... a_m \in L$ con $m \geq$ N. Questo significa che: $q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \xrightarrow{a_3} ... \xrightarrow{a_m} q_m \in F$ ne viene dunque che il camino tocca $m+1$ stati, questo significa che devono esistere i, j tali che $i < j$ tali che $q_i = q_j$ visto che sono passato sullo stesso stato. Per cui possiamo individuare $u = a_1 ... a_i$, $v = a_{i+1} ... a_j$ e $w = a_{j+1} ... a_m$. Poiche' $i < j$ per come ho definito v sicuramente $|v| \geq 1$, la condizione $|uv| \leq N$ mi dice che entro N passi un ciclo si deve essere formato perche' v e' proprio il ciclo essendo composto da tutti gli stati che vanno da q_i a q_j perche abbiamo detto $q_i = q_j$. Infine $\forall k \geq 0 : uv^k w \in L$ e' vero perche' con k andiamo ad indicare sostanzialmente il numero di volte che ripetiamo il ciclo.

3.26 Come si può utilizzare il pumping lemma (a rovescio) per dimostrare che un linguaggio non è regolare?

Se $\forall N > 0$, $\exists z \in L$ con $|z| \geq N$ tale che:

- $\forall u, v, w$, se
  - $z = uvw$
  - $|uv| \leq N$
  - $|v| \geq 1$
- $\exists k \geq 0 : uv^k w \notin L$

allora L non e' regolare.

3.27 Quali sono le proprietà di chiusura dei linguaggi regolari? Quali proprietà si possono decidere (ovvero verificare algoritmicamente)?

La classe dei linguaggi regolari e' chiusa per:

- unione
- concatenazione
- stella di Kleene
- complementazione
- intersezione

Per vedere quelle decisibili aloritimicamente pagina 417 lucidi e appunti.

4 Capitolo 4 - Analisi sintattica - linguaggi liberi

4.1 Cosa si intende per analisi sintattica? Cos'è un parser?

Per analisi sintattica si intende l'analisi della sintassi, ovvero la correttezza della frase. Un parser genera, a partidre dalla lista di token, un albero di derivazione relativo alla grammatica libera usata per costruirlo.

4.2 Definizione di automa a pila nondeterministico (PDA). Definizione di configurazione (o descrizione istantanea), mossa in un passo e mossa in più passi.

Un automa a pila non deterministico e' una 7-upla $(\Sigma, Q, \Gamma, \delta, q_0, \perp, F)$ dove:

- Σ e' l'alfabeto di simboli in input
- Q e' l'insieme di stati
- Γ e' l'insieme finito di simboli della pila
- δ e' la funzione di transizione: $\delta : Q \times (\Gamma \cup \{\epsilon\}) \times \Gamma \to P_{fin}(Q \times \Gamma^*)$
- $q_0 \in Q e' lo stato iniziale$
- $\perp \in \Gamma$ e' il simbolo iniziale della pila
- $F \subseteq Q$ e' l'insieme di stati finali

Mentre l'istantanea e' una tripla (q, w, β) dove

- $q \in Q$ e' lo stato corrente
- $w \in \Gamma^*$ e' l'input non ancora leftmost
- $\beta \in \Gamma^*$ e' la stringa sulla pila

Per la mossa data la sintassi atroce per riportarla guardare pagina 429 di lucidi e appunti.

4.3 Definizione di linguaggio accettato da un PDA per stato finale o per pila vuota. Sono equivalenti queste due modalità di riconoscimento?

Due sono le modalita' di riconoscimento:

- **Stato finale** $L[N] = \{w \in \Gamma^* | (q_0, w, \bot) \vdash_N^* (q, \epsilon, \alpha) \; con \; q \in F\}$
- **Pila vuota** $P[N] = \{w \in \Gamma^* | (q_0, w, \bot) \vdash_N^* (q, \epsilon, \epsilon)\}$

La classe dei linguaggi riconosciuti dai PDA per stato finale e' la stessa di quella dei PDA per pila vuota. Esiste un teorema che ci assicura che:

- Dato un PDA N, se L = P[N] (per pila vuota), possiamo costruire un PDA N' tale che L = L[N'].

  Per costruire N' e' sufficiente aggiungere il simbolo Z alla pila che ne rimane costantemente in fondo e, una volta rimasto solo quello, rimuoverlo con il passo che porta ad un terminale. A livello pratico si tratta di introdurre uno stato iniziale i con passo: $\epsilon, Z/\bot Z$ che si collega al vecchio stato inziale e collegare tutti gli stati dell'automa al nuovo finale con un passo che cancella la Z.

- Dato un PDA N, se L = L[N] (per stato finale), possimo costruire un PDA N' tale che L = P[N'].

  Similmente al caso precendente aggiungo un nuovo stato i come iniziale che scrive sulla pila il simbolo Z di fine pila poi da ogni stato terminale dell'automa creo un passo che porta ad una nuovo stato non finale con un passo che svuota la pila (sia con il passo in se che con un cappio nello stato finale).

4.4 Mostrare come data una grammatica libera G, sia possibile costruire un PDA P equivalente. È possibile costruire, dato un PDA P, una grammatica equivalente G? Concludere che la classe dei linguaggi liberi coincide con la classe dei linguaggi riconosciuti da PDA.

Data una grammatica libera G è possibile costruire un PDA equivalente tramite il metodo *top-down*: data la grammatica si costruisce un automa con un solo stato, con simbolo sulla pila S e che riconosca il linguaggio descritto dalla grammatica per pila vuota. A questo punto le transizioni (che sono sostanzialmente dei cappi) possono essere di due tipi:

- **Espandi**: ovvero una transizione ϵ che sostituisce S in cima alla pila con la parte destra della sua produzione.
- **Consuma**: trasizione che si esegue quando viene fatto match tra l'input e l'ultimo simbolo sulla pila.

Il PDA ottenuto è per forza di cose non deterministico perchè la scelta della transizione espandi è puramente arbitraria. Esiste un teorma che ci dice che un linguggio L è libero da contesto sse è accettato da PDA.

4.5 Quali sono le proprietà di chiusura dei linguaggi liberi? L'intersezione di un linguaggio libero con un regolare è un linguaggio libero?

I linguaggi liberi sono chiusi per: unione, concatenazione e ripetizione/stella di Kleene. Molto più importante è la chiusura rispetto all'intersezione con un linguaggio regolare: esiste infatti un teorema che ci assicura che dato L_1 libero e L_2 regolare allora $L_1 \cap L_2$ è un linguagggio libero. La dimostrazione si fa prendendo N_1 ovvero il PDA che riconosce L_1 per stato finale, N_2 il DFA che riconosce L_2 e crea l'automa prodotto: in pratica si muove sul PDA salvandosi lo stato su cui sarebbe nel DFA e alla fine del riconoscimento di una qualsiasi stringa appartenente al linguaggio intersezione in entrambi gli automi si arriva ad uno stato finale.

4.6 Intestazione e dimostrazione del "Pumping Theorem".

Se L è libero allora $\exists N > 0$ tale che $\forall Z \in L$ con $|z| \geq N, \exists u, v, w, x, y$ tali che:

- $z = uvwxz$
- $|vwz| \leq N$
- $|vx| \geq 1$
- $\forall K \geq uv^k wx^k y \in L$

Sia $G = (NT, T, R, S)$ una grammatica libera tale che $L = L(G)$ e sia b il massimo fattore di ramificazione in un albero di derivazione: $b = max|\alpha|tcA \to \alpha \in R$, con h l'altezza dell'albero (quindi il numero di foglie è b^h). Fisso $N = b^{|NT|+1}$, dunque ogni albero di derivazione per la stringa z ($|z| \geq N$) deve avere altezza almeno di $|NT| + 1$. Preso dunque un qualunque z consideriamo il suo albero di derivazione (nel caso fosse una grammatica ambigua prendiamo quello con il numero minore di nodi). Dunque avendo $|z| \geq N$ il suo albero di derivazione avrà un'altezza maggiore di $|NT| + 1$ dunque il cammino dalla radice S ad una foglia tocca almeno $|NT| + 2$ nodi e $|NT| + 1$ nodi non terminali; ma questo è in contraddizione con l'ipotesi di aver scelto sempre l'albero minimo. Dunque la derivazione $S \Rightarrow^* z$ può essere divisa come: $S \Rightarrow^* uAy \Rightarrow^* uvAxy \Rightarrow^* uvwxy$ con la sotto-derivazione $A \Rightarrow^* vAx$ ripetibile più volte. Quindi anche l'ultima condizione è soddisfatta, basta ora verificare che $|vw| \geq 1$ vero perchè se entrambe ϵ l'albero per $k = 0$ genererebbe ancora z avendo meno nodi e contraddicendo l'ipotesi dell'albero più piccolo e che $|vwx| \leq N$ ovvio perchè il cammino da A alla foglia è minore o uguale a $|NT| + 1$ quindi la A non può genrare parole più lunghe di $b^{|NT|+1} = N$.

4.7 Come si può utilizzare tale teorema (a rovescio) per dimostrare che un linguaggio non è libero?

Se L è libero allora $\forall N > 0$ tale che $\exists Z \in L$ con $|z| \geq N, \forall u, v, w, x, y$ tali che:

- $z = uvwxz$
- $|vwz| \leq N$
- $|vx| \geq 1$
- $\exists K \geq uv^k wx^k y \notin L$

Allora L non è libero

4.8 Classificazione di Chomsky delle grammatiche e dei linguaggi. Definizione di grammatica dipendente dal contesto e di grammatica monotona. Quale tipo di automi corrisponde ad ogni classe?

Slides 465-472 lucidi e appunti

4.9 Definizione di DPDA (automa a pila deterministico) e di linguaggio libero deterministico.

Un PDA $N = (\Sigma, Q, \Gamma, \delta, q_0, \bot, F)$ è un DPDA se valgono le seguenti condizioni:

- $\forall q \in Q, \forall Z \in \Gamma, \forall a \in \Sigma \cup \{\epsilon\}$, vale $|\delta(q, a, Z)| \leq 1$
- $\forall q \in Q, \forall Z \in \Gamma, se\ \delta(q, \epsilon, Z) = \emptyset$, allora $\delta(q, a, Z) = \emptyset \forall a \in \Sigma$

Che, al netto di questa sintassi atroce, può essere scritto come: $|\delta(q, a, Z)| + |\delta(q, \epsilon, Z)| \leq 1$ ovvero che c'è sempre una sola mossa possibile. Un linguaggio libero è deterministico se è accettato per stato finale da un DPDA.

4.10 La classe dei linguaggi liberi deterministici è strettamente inclusa in quella dei linguaggi liberi? Contiene strettamente la classe dei linguaggi regolari?

Un teorema ci dice che la classe dei linguaggi liberi deterministici è inclusa propriamente nella classe dei linguaggi liberi.

4.11 Che cosa dice la prefix property e perché è interessante per i DPDA?

La prefix property dice che un linguaggio deterministico L è riconosciuto da un PDPA per pila vuota sse L è prefix free ovvero $\nexists x, y \in L$ tali che x è prefisso di y.

4.12 Usando un endmarker $, si può riconoscere un linguaggio libero deterministico che non gode della prefix property anche per pila vuota? Come?

Il trucco che rende possibile riconoscere un linguaggio libero deterministico tramite un DPDA è quello di aggiungere un endmarker alla fine di ogni stringa in modo tale da rendere impossibile che un'altra stringa del linguaggio ne sia prefisso.

4.13 Un linguaggio libero deterministico è ambiguo?

No, un linguaggio libero non e' ambiguo perche' esso e' generabile da una grammatica libera non ambigua (proposizione).

4.14 Proprietà di chiusura dei linguaggi liberi deterministici: chiusi per complementazione, ma non per intersezione né per unione.

I linguagggi liberi appunto sono chiusi per complementazione e l'intersezione tra un linguaggio libero deterministico e uno regolare e' un linguaggio a sua volta libero deterministico. Invece, non sono chiusi per intersezione, per unione e concatenzaione/stella di Kleene.

4.15 Da cosa si parte per costruire un analizzatore sintattico (ovvero parser)? Da una espressione regolare? Da una grammatica libera? Da un PDA?

Si parte da un PDA.

4.16 Cosa prende in input e cosa produce in output un parser?

Un parser prende in input una lista di token e produce un albero di derivazione.

4.17 Quali sono le due tecniche essenziali per costruire parsers?

Le due tecniche essenziali per costruire un parser sono Top-down e Bottom-up.

4.18 Le tecniche top-down e bottom-up in che cosa differiscono?

- **Top-down**: ricostruiscono una derivazione left-most per w a partire da S.
- **Bottom-up**: ricostruiscono una derivazione right-most per w a partire da S.

4.19 Quali tipi di grammatiche non sono adatte al top-down parsing? Quali tipi di produzioni sono poco adatte al bottom-up parsing?

Le grammatiche left recursive sono quelle meno adatte al top-down parsing perchè mantendo ricorsivamente S il piu' a sinistra possibile finiscono per bloccare la prima meta' della parola (sl. 501). Invece le grammatiche libere con produzioni ϵ sono quelle meno adatte al botto-upm parsing.

4.20 Cosa sono le produzioni epsilon e cosa sono i simboli non-terminali annullabili?

Le produzioni epsilon sono le produzioni del tipo $A \to \epsilon$ che sono poco adatte al bottom-up parsing. Mentre un simbolo non terminale è annullabile se $A \in NT | A \Rightarrow^+ \epsilon$.

4.21 Come si può trasformare una grammatica G che contiene produzioni epsilon in una grammatica G' che non ne contiene, preservando il linguaggio a meno di epsilon?

Per trasformare una grammatica G che contiene produzioni epsilon in una grammatica G' che non ne contiene è sufficiente formare l'insieme $N(G)$ che contiene tutti i simboli annullabili e considerare, per ogni elemento, tutti i modi in cui si possono cancellare i simboli annullabili. Esempio a pagina 520 lucidi e appunti.

4.22 Cosa sono le produzioni unitarie e cosa sono le coppie unitarie?

Le produzioni unitarie sono produzioni del tipo $A \to B$ con $A, B \in NT$, mentre una coppia unitaria è formata da non terminali tali che $A \Rightarrow^* B$ usando solo produzioni unitarie.

4.23 Come si può trasformare una grammatica G che contiene produzioni unitarie in una equivalente G' che non ne contiene?

Data la grammatica G libera per costruire la grammatica G' per ogni coppia unitaria $((A, B) \in U(G))$ e per ogni $B \to \alpha \in R$ metto in R' la produzione $A \to \alpha$. Esempio a pagina 526 lucidi e appunti.

4.24 Data una grammatica G, quali sono i suoi simboli utili? Quali sono i suoi simboli generatori? Quali i suoi simboli raggiungibili?

Un simbolo $X \in T \cup NT$ è:

- Generatore sse $\exists w \in T^*$ con $X \Rightarrow^* w$.
- Raggiungibile sse $S \Rightarrow^* \alpha X \beta$ per qualche $\alpha, \beta \in (T \cup NT)$.
- Utile sse è generatore e raggiungibile.

4.25 Come si calcolano i generatori? Come si calcolano i raggiungibili?

Data la grammatica G per calcolare i generatori è sufficiente calcolare quali terminali riescono a produrre dei terminali in modo induttivo per ottenere l'insieme dei generatori, una volta eliminati dalla grammatica tutti i terminali raggiungibili in modo induttivo guardando tutti i non terminali che riesco a ragiungere, partendo da S, in un numero i di passi.

4.26 In che modo si eliminano i simboli inutili di una grammatica? È importante l'ordine delle operazioni da svolgere?

Per eliminare i simboli inutili di una grammatica prima si trovano i simboli non generatori e si eliminano tutte le produzioni che contengono almeno uno di questi simboli, ssuccessivamente calcolo tutti i simboli non raggiungibili ed elimino tutte le produzioni che ne contengono. L'ordine di esecuzione delle due fasi è vitale, la grammatica risultante potrebbe essere errata.

4.27 Quando si dice che una grammatica è ricorsiva sinistra? Come si elimina la ricorsione sinistra immediata? E quella non immediata? Perché serve eliminare la ricorsione sinistra?

UNa grammatica si dice ricorsiva sinistra se ha una qualche produzione ricorsiva sinistra; una produzione è ricorsiva sinistra se è del tipo $A \to a\alpha$. Per eliminare la ricorsione sinistra immediata data la produzione: $A \to A\alpha_1|...|A\alpha_n|\beta_\|...|\beta_n$ è sufficiente rimpiazzare le produzioni nel seguente modo:

$$A \to \beta_1 A'|...|\beta_n A'$$
$$A' \to \alpha_1 A'|...|\alpha_n A'|\epsilon$$

Esempio pagina 546 lucidi e appunti.

Per eliminare la ricorsione sinistra non immediata si deve avere innanzi tutto una gramma tica libera senza ϵ-produzioni e senza produzioni unitarie se no l'algoritmo non funziona. L'algoritmo sostanzialmente funziona andando ad analizzare un non-terminale alla volte e, una volta indivuduata la produzione che genera ricorsione sinistra non immediata, rimpiazzare il non terminale incriminato con le sue produzioni. A questo punto, se necessario, si elimina la ricorsione sinistra immediata. E' necessario eliminare la ricorsione sinistra perchè non e' adatta al top-down parsing.

4.28 Cosa vuol dire fattorizzare a sinistra una grammatica? Perché serve fattorizzare?

Fattorizzarte a sinistra una grammatica significa raccogliere il prefisso piu' lungo delle produzioni della stessa, questo procedimento si fa per evitare il non-determinismo iniseme al look-ahead.

4.29 Definizione di First(a) e di Follow(A).

Con $first(a)$ intendiamo tutti i simboli terminali che possono stare all'inizio di una produzione derviata da a, mentre con $Follo(A)$ intendiamo tutti i simboli che possono essere direttamente a destra del simbolo A in una derivazione a partire da S.

4.30 Algoritmi per calcolare First(a) e di Follow(A).

- **first**: per determiare il first bisogna prima vedere se si ha la forma αA oppure $A\alpha$, nel primo caso il first e' proprio α essendo un non terminale. Nel secondo caso il first e' il first delle produzioni di A (l'insieme dei first puo' essere composto da piu' elementi).
- **follow**: il follow di tutte le sentenze e' di base \$, per espandere l'insieme basta applicare le regole
  - Per ogni produzione $X \to \alpha Y \beta$ $Follow(Y) := Follow(Y) \cup (First(\beta) - \{\epsilon\})$
  - Per ogni produzione $X \to \alpha Y$ oppure $X \to \alpha Y \beta$ con $\epsilon \in First(\beta)$ $Follow(Y) := Follow(Y) \cup Follow(X)$

4.31 **Come è fatta e come si riempie una tabella di parsing LL(1)?**

4.32 **Quando una grammatica G si dice di classe LL(1)? Quali sono le condizioni necessarie e sufficienti per G affinché sia di classe LL(1)?**

4.33 **Perché un parser di questo tipo è chiamato LL?**

4.34 **Come funziona il parser LL(1) con una pila?**

4.35 ***Cos'è un parser a discesa ricorsiva?***

4.36 **È vero che ogni linguaggio regolare è pure LL(1)?**

4.37 **Come sono definiti Firstk(a) e di Followk (A) per $k \geq 2$.**

4.38 **Come si definisce una grammatica LL(k)? Ed un linguaggio LL(k)? Come si relazionano tra di loro?**

4.39 **Che relazione esiste tra grammatiche LL(k) e grammatiche ambigue? E con le grammatiche ricorsive sinistre?**

4.40 **Esistono linguaggi liberi che non sono LL(k) per nessun k? Ed esistono linguaggi liberi deterministici che non sono LL(k) per nessun k?**

4.41 **Cos'è un parser bottom-up (o shift-reduce)? Qual è il suo input e il suo output? Perché sono chiamati parser LR?**

4.42 **Che tipo di conflitti si possono presentare in un parser del genere? Quando si presenta un conflitto (shift/reduce o reduce/reduce), quale azione bisogna scegliere?**

4.43 ***Cos'è un prefisso viabile? Come lo si definisce in termini di una grammatica?***

4.44 ***Cos'è un item LR(0)? Come si generano tutti gli item di una grammatica (aumentata con un simbolo iniziale nuovo S')?***

4.45 **Come si ricava il DFA dei prefissi viabili, detto anche automa canonico LR(0)?**

4.46 **Come è fatta una tabella di parsing LR(0)? Come la si riempie a partire dall'automa canonico LR(0)? Quando una grammatica è di classe LR(0)?**

4.47 **Come è fatto il parser LR(0) che utilizza la tabella di parsing LR(0)? Quanti stack servono?**

4.48 **Esistono grammatiche non LR(0)? Fare un esempio semplice.**

4.49 **Come è fatta e come si riempie una tabella di parsing SLR(1)? Cosa vuol dire l'acronimo SLR? Perché si mette 1 come parametro?**

4.50 **Quando una grammatica è di classe SLR(1)? Esistono grammatiche non di classe SLR(1)?**

4.51 ***Cos'è un item LR(1)? Come si costruisce l'automa canonico LR(1)?***

4.52 ***Come è fatta e come si riempie una tabella di parsing LR(1)?***

4.53 ***Come è fatto una tabella di parsing LALR(1)? Quando una grammatica è di classe LALR(1)?***

4.54 **Esistono grammatiche LALR(1) che non sono SLR(1)? Esistono grammatiche LR(1) che non sono LALR(1)? Mostrare una grammatica che è LR(1) ma non LALR(1).**

4.55 **Come si può generalizzare l'idea per ogni $k \geq 2$? Ovvero quando una grammatica è di classe LR(k), SLR(k) o LALR(k)?**

4.56 **Come si relazionano le grammatiche della famiglia LR (LR, SLR, LALR) al variare di k? Come si relazionano le grammatiche LR(k) e LL(k) al variare di k?**

4.57 **Le grammatiche LR(k) e LL(k) sono sempre non ambigue? Esistono grammatiche**